Vlasov-1D Normalization Derivation: Wave Equation, Vlasov Forces, Ponderomotive Force, and Field Solvers

ADEPT Documentation

March 11, 2026

Contents

1 Normalization Convention **3**
- 1.1 Reference Quantities . . . 3
- 1.2 Dimensionless Variables . . . 3
- 1.3 Normalized Charges and Masses . . . 3
- 1.4 Normalized Fields . . . 3

2 The Vlasov Equation and `edfdv` Push **4**
- 2.1 SI Vlasov Equation . . . 4
- 2.2 Normalized Form . . . 4
- 2.3 Verification Against Code (`VelocityExponential`) . . . 4

3 Poisson Solver **5**
- 3.1 SI Poisson Equation . . . 5
- 3.2 Normalization . . . 5
- 3.3 Verification Against Code (`SpectralPoissonSolver`) . . . 5

4 Ampere Solver **6**
- 4.1 SI Ampere's Law (1D Electrostatic) . . . 6
- 4.2 Normalization . . . 6
- 4.3 Verification Against Code (`AmpereSolver`) . . . 6
- 4.4 Hamiltonian Ampere Solver (`HampereSolver`) . . . 6

5 Wave Equation for the Transverse Vector Potential **7**
- 5.1 SI Wave Equation . . . 7
- 5.2 Normalization . . . 7
- 5.3 Normalized Wave Equation . . . 7
- 5.4 Verification Against Code (`WaveSolver`) . . . 7
  - 5.4.1 The electron density term . . . 8
  - 5.4.2 The source term `djy_array` . . . 8

6 Ponderomotive Force **9**
6.1 SI Derivation for General Species . . . 9
6.2 Normalization . . . 9
6.3 Expressing as an Effective Electric Field . . . 9
6.4 Verification Against Code . . . 10

7 The `ex_driver` — Longitudinal Electric Field Source **11**
7.1 Physical Setup . . . 11
7.2 Derivation . . . 11
7.3 Comparison with Current Code . . . 11

8 The `ey_driver` — Transverse Current Source for the Wave Equation **12**
8.1 Physical Setup . . . 12
8.2 Derivation: Required External Current . . . 12
8.3 Interpretation . . . 12
8.4 Comparison with Current Code . . . 13

9 Summary **14**
9.1 Components Verified Correct . . . 14
9.2 Bugs Found . . . 14
9.3 Recommended Fixes . . . 14
9.3.1 Ponderomotive force (multi-species correct) . . . 14
9.3.2 Driver classes . . . 14

1 Normalization Convention

1.1 Reference Quantities

The Vlasov-1D module uses electron-Debye normalization. All quantities are measured relative to:

$$\tau = \frac{1}{\omega_{p0}}, \qquad \omega_{p0} = \sqrt{\frac{n_0 e^2}{m_e \varepsilon_0}} \qquad \text{(reference time = inverse plasma frequency)} \tag{1}$$

$$v_0 = \sqrt{\frac{2T_0}{m_e}} \qquad \text{(electron thermal velocity)} \tag{2}$$

$$L_0 = \frac{v_0}{\omega_{p0}} \qquad \text{(Debye length)} \tag{3}$$

$$c_{\text{norm}} = \frac{c}{v_0} \qquad \text{(normalized speed of light)} \tag{4}$$

In the code, $\beta = v_0/c = 1/c_{\text{norm}}$.

1.2 Dimensionless Variables

$$\tilde{x} = \frac{x}{L_0}, \qquad \tilde{t} = \frac{t}{\tau} = \omega_{p0} t, \qquad \tilde{v} = \frac{v}{v_0} \tag{5}$$

$$\tilde{k} = k L_0, \qquad \tilde{\omega} = \frac{\omega}{\omega_{p0}} \tag{6}$$

1.3 Normalized Charges and Masses

For species s with SI charge q_s and SI mass m_s, define:

$$\tilde{q}_s \equiv \frac{q_s}{e}, \qquad \tilde{m}_s \equiv \frac{m_s}{m_e} \tag{7}$$

For electrons: $\tilde{q}_e = -1$, $\tilde{m}_e = 1$. For ions: $\tilde{q}_i = +Z$, $\tilde{m}_i = M/m_e$.

In the code, these are `charge` and `mass` respectively, with `charge_to_mass` $= \tilde{q}_s/\tilde{m}_s$.

1.4 Normalized Fields

The **normalized electric field**:

$$\boxed{\tilde{E} = \frac{eE}{m_e v_0 \omega_{p0}}} \tag{8}$$

The electron equation of motion $m_e \frac{dv}{dt} = -eE$ becomes $\frac{d\tilde{v}}{d\tilde{t}} = -\tilde{E}$, or more generally for species s: $\frac{d\tilde{v}}{d\tilde{t}} = \frac{\tilde{q}_s}{\tilde{m}_s}\tilde{E}$.

The **normalized vector potential**:

$$\boxed{\tilde{a} = \frac{eA}{m_e v_0}} \tag{9}$$

Note the relationship to the standard relativistic $a_0^{\text{std}} = eA_0/(m_e c)$:

$$\tilde{a}_0 = a_0^{\text{std}} \cdot c_{\text{norm}} \tag{10}$$

The **normalized density**: $\tilde{n}_s = n_s/n_0$.

The **normalized distribution function** satisfies $\int f_s \, d\tilde{v} = \tilde{n}_s$.

2 The Vlasov Equation and `edfdv` Push

2.1 SI Vlasov Equation

For species s with charge q_s and mass m_s:

$$\frac{\partial f_s}{\partial t} + v\frac{\partial f_s}{\partial x} + \frac{q_s E}{m_s}\frac{\partial f_s}{\partial v} = 0 \tag{11}$$

2.2 Normalized Form

Substituting $t = \tilde{t}/\omega_{p0}$, $x = \tilde{x}L_0$, $v = \tilde{v}v_0$, $E = (m_e v_0 \omega_{p0}/e)\tilde{E}$:

$$\frac{\partial f_s}{\partial \tilde{t}} + \tilde{v}\frac{\partial f_s}{\partial \tilde{x}} + \underbrace{\frac{q_s}{m_s}\cdot\frac{m_e}{e}}_{\tilde{q}_s/\tilde{m}_s}\cdot\tilde{E}\frac{\partial f_s}{\partial \tilde{v}} = 0 \tag{12}$$

The normalized Vlasov equation is:

$$\boxed{\frac{\partial f_s}{\partial \tilde{t}} + \tilde{v}\frac{\partial f_s}{\partial \tilde{x}} + \frac{\tilde{q}_s}{\tilde{m}_s}\,\tilde{E}_{\text{total}}\frac{\partial f_s}{\partial \tilde{v}} = 0} \tag{13}$$

The acceleration of species s is:

$$\tilde{a}_s = \frac{\tilde{q}_s}{\tilde{m}_s}\,\tilde{E}_{\text{total}} \tag{14}$$

This is the key relationship: $\tilde{E}$ is *not* the acceleration—it must be multiplied by $\tilde{q}_s/\tilde{m}_s$ to give the acceleration. Every quantity added to `e` in the code must therefore be in units of $\tilde{E}$, not acceleration.

2.3 Verification Against Code (`VelocityExponential`)

The exponential pusher in `vlasov.py:63--71` computes:

```
qm = species_params[species_name]["charge_to_mass"]    # = tilde_q /
    tilde_m
result = irfft(exp(-i * kv * dt * qm * e) * rfft(f))
```

This shifts the distribution in velocity space by $\Delta\tilde{v} = (\tilde{q}_s/\tilde{m}_s)\cdot\tilde{E}\cdot\Delta\tilde{t}$, consistent with eq. (14). Note the per-species $\tilde{q}_s/\tilde{m}_s$ factor is applied correctly.

Conclusion: The `edfdv` pusher is correctly normalized *provided* everything passed as `e` is in units of $\tilde{E}$ as defined by eq. (8). The three additive contributions to `e` are:

1. `pond` — ponderomotive force (Section 6)
2. `e` — self-consistent electrostatic field from Poisson/Ampere (Section 3)
3. `dex_array[0]` — external driver field (Section 7)

All three must carry units of $\tilde{E}$ (electric field), **not** acceleration. The $\tilde{q}_s/\tilde{m}_s$ conversion to acceleration is handled inside the `edfdv` push.

3 Poisson Solver

3.1 SI Poisson Equation

$$\nabla^2\phi = -\frac{\rho}{\varepsilon_0}, \qquad E = -\nabla\phi \tag{15}$$

where $\rho = \sum_s q_s n_s$ is the total charge density.

3.2 Normalization

Define $\tilde{\phi} = e\phi/(m_e v_0^2)$ so that $\tilde{E} = -\partial\tilde{\phi}/\partial\tilde{x}$.

Verification: In SI, $E = -\partial_x\phi$. So:

$$\tilde{E} = \frac{eE}{m_e v_0 \omega_{p0}} = -\frac{e}{m_e v_0 \omega_{p0}} \cdot \frac{1}{L_0}\frac{\partial\phi}{\partial\tilde{x}} = -\frac{e}{m_e v_0^2}\frac{\partial\phi}{\partial\tilde{x}} = -\frac{\partial\tilde{\phi}}{\partial\tilde{x}} \quad \checkmark \tag{16}$$

using $L_0 = v_0/\omega_{p0}$.

Now normalize the Poisson equation. Define $\tilde{\rho} = \rho/(en_0) = \sum_s \tilde{q}_s\tilde{n}_s$. The SI equation $\partial_{\tilde{x}}^2\phi = -\rho/\varepsilon_0$ becomes:

$$\frac{1}{L_0^2}\frac{\partial^2}{\partial\tilde{x}^2}\left(\frac{m_e v_0^2}{e}\tilde{\phi}\right) = -\frac{en_0}{\varepsilon_0}\tilde{\rho} \tag{17}$$

The LHS prefactor is $m_e v_0^2/(eL_0^2) = m_e\omega_{p0}^2/e = n_0 e/\varepsilon_0$, which equals the RHS prefactor. Therefore:

$$\boxed{\frac{\partial^2\tilde{\phi}}{\partial\tilde{x}^2} = -\tilde{\rho}} \tag{18}$$

In Fourier space: $\hat{\tilde{E}}_k = -i\tilde{k}\hat{\tilde{\phi}}_k = \frac{-i}{\tilde{k}}\hat{\tilde{\rho}}_k$.

3.3 Verification Against Code (`SpectralPoissonSolver`)

The code (`field.py:135--160`):

```
rho = sum_s q_s * sum(f_s, axis=1) * dv_s  # + static_charge
E = real(ifft(-1j * one_over_kx * fft(rho)))
```

Here `q_s` $= \tilde{q}_s$ is the normalized charge, `sum(f_s)*dv_s` $= \tilde{n}_s$, so `rho` $= \sum_s \tilde{q}_s\tilde{n}_s = \tilde{\rho}$. The spectral operation gives $\hat{\tilde{E}}_k = -i(1/\tilde{k})\hat{\tilde{\rho}}_k$.

Result: The Poisson solver is correctly normalized. Its output is $\tilde{E}$ in the correct units for the Vlasov push.

4 Ampere Solver

4.1 SI Ampere's Law (1D Electrostatic)

In 1D electrostatics:

$$\varepsilon_0 \frac{\partial E}{\partial t} = -j, \qquad j = \sum_s q_s \int v f_s \, dv \tag{19}$$

4.2 Normalization

Substitute $E = (m_e v_0 \omega_{p0}/e)\tilde{E}$, $t = \tilde{t}/\omega_{p0}$, and define $\tilde{j} = \sum_s \tilde{q}_s \int \tilde{v} f_s \, d\tilde{v}$:

$$\varepsilon_0 \cdot \frac{m_e v_0 \omega_{p0}}{e} \cdot \omega_{p0} \frac{\partial \tilde{E}}{\partial \tilde{t}} = -e n_0 v_0 \tilde{j} \tag{20}$$

The LHS prefactor is $\varepsilon_0 m_e v_0 \omega_{p0}^2/e$. Using $\omega_{p0}^2 = n_0 e^2/(m_e \varepsilon_0)$:

$$\frac{\varepsilon_0 m_e v_0}{e} \cdot \frac{n_0 e^2}{m_e \varepsilon_0} = e n_0 v_0 \tag{21}$$

So the normalized Ampere equation is:

$$\boxed{\frac{\partial \tilde{E}}{\partial \tilde{t}} = -\tilde{j}, \qquad \tilde{j} = \sum_s \tilde{q}_s \int \tilde{v} f_s \, d\tilde{v}} \tag{22}$$

4.3 Verification Against Code (`AmpereSolver`)

The code (`field.py:192--216`):

```
j_s = sum(v * f_s, axis=1) * dv_s    # integral of v*f dv for species s
j += q_s * j_s                        # q_s = tilde_q
return prev_ex - dt * j
```

Here `q_s` $= \tilde{q}_s$, so `j` $= \sum_s \tilde{q}_s \int \tilde{v} f_s \, d\tilde{v} = \tilde{j}$. The update implements $\tilde{E}^{n+1} = \tilde{E}^n - \Delta\tilde{t} \cdot \tilde{j}$.

Result: The Ampere solver is correctly normalized. Its output is $\tilde{E}$.

4.4 Hamiltonian Ampere Solver (`HampereSolver`)

The Hamiltonian Ampere solver integrates Ampere's law exactly along ballistic characteristics in Fourier space (`field.py:273--278`):

```
new_ek = prev_ek + charge * one_over_ikx
         * sum(fk * (exp(-i*kx*dt*vx) - 1), axis=1) * dv
```

This implements:

$$\hat{\tilde{E}}_k(\tilde{t} + \Delta\tilde{t}) = \hat{\tilde{E}}_k(\tilde{t}) + \frac{\tilde{q}_s}{i\tilde{k}} \int \hat{f}_{s,k}(\tilde{t}) \left(e^{-i\tilde{k}\tilde{v}\Delta\tilde{t}} - 1 \right) d\tilde{v} \tag{23}$$

which is the exact solution of $\partial_{\tilde{t}} \hat{\tilde{E}}_k = -\hat{\tilde{j}}_k$ with free-streaming f.

Result: The Hamiltonian Ampere solver is correctly normalized.

5 Wave Equation for the Transverse Vector Potential

5.1 SI Wave Equation

The 1D wave equation for the transverse vector potential A_y in a plasma, driven by an external transverse current $j_{\text{ext},y}$:

$$\frac{\partial^2 A_y}{\partial t^2} = c^2 \frac{\partial^2 A_y}{\partial x^2} - \frac{n_e e^2}{m_e \varepsilon_0} A_y + \frac{j_{\text{ext},y}}{\varepsilon_0} \tag{24}$$

The $-\frac{n_e e^2}{m_e \varepsilon_0} A$ term is the linearized transverse quiver current. Note $\frac{e^2 n_0}{m_e \varepsilon_0} = \omega_{p0}^2$.

5.2 Normalization

Substitute $A_y = (m_e v_0/e)\tilde{a}$, $t = \tilde{t}/\omega_{p0}$, $x = \tilde{x} L_0$. Every term acquires the common prefactor $\frac{m_e v_0}{e}\omega_{p0}^2$:

LHS: $\frac{m_e v_0}{e}\omega_{p0}^2 \frac{\partial^2 \tilde{a}}{\partial \tilde{t}^2}$

$c^2 \partial_x^2 A$ **term:** Uses $c^2/L_0^2 = c_{\text{norm}}^2 \omega_{p0}^2$ to give $\frac{m_e v_0}{e}\omega_{p0}^2 c_{\text{norm}}^2 \frac{\partial^2 \tilde{a}}{\partial \tilde{x}^2}$.

Density term: $-\frac{n_e}{n_0}\omega_{p0}^2 \cdot \frac{m_e v_0}{e}\tilde{a} = \frac{m_e v_0}{e}\omega_{p0}^2 \left(-\frac{n_e}{n_0}\right)\tilde{a}$.

External current term: We need to express $j_{\text{ext},y}/\varepsilon_0$ in the same units. The common prefactor is $\frac{m_e v_0}{e}\omega_{p0}^2$, so define:

$$\tilde{S} = \frac{j_{\text{ext},y}}{\varepsilon_0} \cdot \frac{e}{m_e v_0 \omega_{p0}^2} \tag{25}$$

Using $\omega_{p0}^2 = n_0 e^2/(m_e \varepsilon_0)$:

$$\tilde{S} = \frac{j_{\text{ext},y}}{\varepsilon_0} \cdot \frac{e}{m_e v_0} \cdot \frac{m_e \varepsilon_0}{n_0 e^2} = \frac{j_{\text{ext},y}}{n_0 e v_0} \tag{26}$$

5.3 Normalized Wave Equation

Dividing through by the common factor $\frac{m_e v_0}{e}\omega_{p0}^2$:

$$\boxed{\frac{\partial^2 \tilde{a}}{\partial \tilde{t}^2} = c_{\text{norm}}^2 \frac{\partial^2 \tilde{a}}{\partial \tilde{x}^2} - \frac{n_e}{n_0}\tilde{a} + \tilde{S}} \tag{27}$$

where

$$\tilde{S} = \frac{j_{\text{ext},y}}{n_0 e v_0} \tag{28}$$

5.4 Verification Against Code (`WaveSolver`)

The wave solver (`field.py:85--88`):

```
anew = 2*a[1:-1] - aold[1:-1]
     + dt**2 * (c_sq*d2dx2 - electron_density*a[1:-1] + djy_array[1:-1])
```

This is a leapfrog discretization of:

$$\frac{\partial^2 \tilde{a}}{\partial \tilde{t}^2} = c_{\text{norm}}^2 \frac{\partial^2 \tilde{a}}{\partial \tilde{x}^2} - n_e^{\text{code}} \cdot \tilde{a} + \text{djy} \tag{29}$$

where `c_sq` $= c_{\text{norm}}^2$ (since `c = 1/beta` $= c_{\text{norm}}$).

5.4.1 The electron density term

The code passes (`vector_field.py:337`):

```
electron_density = -0.5*(electron_charge_density_n +
    electron_charge_density_np1)
```

where `electron_charge_density` is computed as (`vector_field.py:296`):

```
charge_density = charge * sum(f, axis=1) * dv    # charge = tilde_q = -1
    for e-
```

So `electron_charge_density` $= \tilde{q}_e \cdot \tilde{n}_e = (-1) \cdot \tilde{n}_e$, and:

$$n_e^{\text{code}} = -\tfrac{1}{2}\big((-\tilde{n}_e^n) + (-\tilde{n}_e^{n+1})\big) = +\tfrac{1}{2}(\tilde{n}_e^n + \tilde{n}_e^{n+1}) \tag{30}$$

The wave equation term is $-n_e^{\text{code}} \cdot \tilde{a} = -\frac{1}{2}(\tilde{n}_e^n + \tilde{n}_e^{n+1})\tilde{a}$. This matches eq. (27) with Crank-Nicolson time-averaging.

Result: The electron density term in the wave equation is correctly normalized.

5.4.2 The source term `djy_array`

Comparing eq. (29) with eq. (27), the code's `djy_array` must equal $\tilde{S} = j_{\text{ext},y}/(n_0 e v_0)$. See Section 8 for the derivation of what the `ey_driver` should produce.

6 Ponderomotive Force

6.1 SI Derivation for General Species

For species s with charge q_s and mass m_s, the quiver kinetic energy in a transverse field A_y is (from the canonical Hamiltonian with $p_y = 0$):

$$K_{\perp,s} = \frac{q_s^2 A_y^2}{2m_s} \tag{31}$$

This acts as a ponderomotive potential. The ponderomotive force is:

$$F_{\text{pond},s} = -\frac{\partial K_{\perp,s}}{\partial x} = -\frac{q_s^2}{2m_s}\frac{\partial(A_y^2)}{\partial x} \tag{32}$$

The ponderomotive *acceleration* is:

$$a_{\text{pond},s} = \frac{F_{\text{pond},s}}{m_s} = -\frac{q_s^2}{2m_s^2}\frac{\partial(A_y^2)}{\partial x} \tag{33}$$

Important: The coefficient q_s^2/m_s^2 is species-dependent. The ponderomotive acceleration is *not* proportional to q_s/m_s (the charge-to-mass ratio that appears in $\vec{F} = q\vec{E}$). It is proportional to q_s^2/m_s^2.

Note on cycle averaging: Since the wave solver evolves the full instantaneous $A_y(x,t)$ (not an envelope), the factor of 1/2 here is correct. The often-seen 1/4 arises from cycle-averaging $\langle \sin^2 \rangle = 1/2$ of the envelope amplitude.

6.2 Normalization

Normalize the acceleration using $\tilde{a}_s = a_s/(v_0\omega_{p0})$, $x = L_0\tilde{x}$, $A_y = (m_e v_0/e)\tilde{a}$:

$$\tilde{a}_{\text{pond},s} = \frac{1}{v_0\omega_{p0}}\left(-\frac{q_s^2}{2m_s^2}\right)\frac{1}{L_0}\frac{\partial}{\partial\tilde{x}}\left(\frac{m_e v_0}{e}\right)^2\tilde{a}^2 \tag{34}$$

$$= -\frac{q_s^2}{2m_s^2}\cdot\frac{m_e^2 v_0}{e^2\omega_{p0}L_0}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}} \tag{35}$$

$$= -\frac{q_s^2 m_e^2}{2m_s^2 e^2}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}} \qquad \text{(using } L_0 = v_0/\omega_{p0}\text{)} \tag{36}$$

$$= -\frac{\tilde{q}_s^2}{2\tilde{m}_s^2}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}} \tag{37}$$

6.3 Expressing as an Effective Electric Field

The `edfdv` push computes the acceleration as $\tilde{a}_s = (\tilde{q}_s/\tilde{m}_s)\cdot\tilde{E}$ (eq. 14). To use this machinery, we need an effective electric field $\tilde{E}_{\text{pond},s}$ such that:

$$\frac{\tilde{q}_s}{\tilde{m}_s}\cdot\tilde{E}_{\text{pond},s} = -\frac{\tilde{q}_s^2}{2\tilde{m}_s^2}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}} \tag{38}$$

Solving:

$$\boxed{\tilde{E}_{\text{pond},s} = -\frac{\tilde{q}_s}{2\tilde{m}_s}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}}} \tag{39}$$

This is species-dependent. The effective ponderomotive "electric field" depends on the charge and mass of the species being pushed.

For electrons ($\tilde{q}_e = -1$, $\tilde{m}_e = 1$):

$$\tilde{E}_{\text{pond},e} = +\frac{1}{2}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}} \tag{40}$$

6.4 Verification Against Code

The code (`field.py:346`):

```
ponderomotive_force = -0.5 * jnp.gradient(a**2.0, self.dx)[1:-1]
```

This computes $-\frac{1}{2}\partial_{\tilde{x}}(\tilde{a}^2)$.

This value is then added directly to `e` in the Vlasov push (`vector_field.py:88`):

```
f_dict = self.edfdv(f_after_v, e=pond + e + dex_array[0], dt=self.dt)
```

The `edfdv` push multiplies by $\tilde{q}_s/\tilde{m}_s$, giving acceleration:

$$\tilde{a}_s^{\text{code}} = \frac{\tilde{q}_s}{\tilde{m}_s} \cdot \underbrace{\left(-\frac{1}{2}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}}\right)}_{\text{pond (code)}} \tag{41}$$

For electrons ($\tilde{q}_e/\tilde{m}_e = -1$):

$$\tilde{a}_e^{\text{code}} = (-1) \cdot \left(-\frac{1}{2}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}}\right) = +\frac{1}{2}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}} \tag{42}$$

But the correct electron ponderomotive acceleration from eq. (37) is:

$$\tilde{a}_e^{\text{correct}} = -\frac{(-1)^2}{2(1)^2}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}} = -\frac{1}{2}\frac{\partial(\tilde{a}^2)}{\partial\tilde{x}} \tag{43}$$

Bug: Sign error in ponderomotive force.
The code computes the ponderomotive *acceleration* $-\frac{1}{2}\partial_{\tilde{x}}(\tilde{a}^2)$ but passes it as an *electric field* to the `edfdv` push, which multiplies by $\tilde{q}_s/\tilde{m}_s$ again. For electrons, this flips the sign: particles are pushed *toward* high intensity instead of away from it.
The correct effective electric field for the ponderomotive force is species-dependent: $\tilde{E}_{\text{pond},s} = -\frac{\tilde{q}_s}{2\tilde{m}_s}\partial_{\tilde{x}}(\tilde{a}^2)$.
For a single-species electron simulation, the fix is to flip the sign: `pond = +0.5 * gradient(a**2, dx)`.
For multi-species, the ponderomotive contribution must be applied per-species inside the `edfdv` loop, not as a universal additive field.

Sanity check: Consider $\tilde{a}^2$ increasing in $+\tilde{x}$ (intensity gradient pointing right). The ponderomotive force should push electrons to the left (away from high intensity).

- **Code:** $\tilde{a}_e = +\frac{1}{2} \cdot (\text{positive}) > 0$ — pushes electrons *right*, toward high intensity. Wrong.
- **Correct:** $\tilde{a}_e = -\frac{1}{2} \cdot (\text{positive}) < 0$ — pushes electrons *left*, away from high intensity. Correct.

7 The `ex_driver` — Longitudinal Electric Field Source

7.1 Physical Setup

The `ex_driver` injects an external longitudinal electric field E_x that is added directly to the total field in the Vlasov push. The driver specifies a wave via amplitude $\tilde{a}_0$, wavenumber $\tilde{k}$, and frequency $\tilde{\omega}$.

7.2 Derivation

If the driver represents an EM wave with vector potential $A = A_0 \sin(kx - \omega t)$, the associated electric field is $E = -\partial_t A = \omega A_0 \cos(kx - \omega t)$.

Normalizing (using $\tilde{E} = eE/(m_e v_0 \omega_{p0})$ and $\tilde{a}_0 = eA_0/(m_e v_0)$):

$$\tilde{E} = \frac{\omega}{\omega_{p0}} \cdot \frac{eA_0}{m_e v_0} \cos\left(\tilde{k}\tilde{x} - \tilde{\omega}\tilde{t}\right) = \tilde{\omega} \cdot \tilde{a}_0 \cos\left(\tilde{k}\tilde{x} - \tilde{\omega}\tilde{t}\right) \tag{44}$$

The sin vs cos difference is a $\pi/2$ phase shift in the definition of the carrier. Writing sin to match the code convention:

$$\boxed{\tilde{E}_{\text{ex,driver}} = \tilde{\omega} \cdot \tilde{a}_0 \cdot \sin\left(\tilde{k}\tilde{x} - \tilde{\omega}\tilde{t}\right)} \tag{45}$$

7.3 Comparison with Current Code

The current `Driver.get_this_pulse` (`field.py:23`):

```
return factor * jnp.abs(kk) * this_pulse.a0 * jnp.sin(kk*x - ww*t)
```

This uses $|\tilde{k}|$ where it should use $\tilde{\omega}$.

Discrepancy: The `ex_driver` is missing a factor of $\tilde{\omega}/|\tilde{k}|$. For vacuum dispersion ($\omega = ck$), this ratio is $c_{\text{norm}} = c/v_0 \gg 1$.
The driver should use $\tilde{\omega}$ directly, which is correct for any dispersion relation.

8 The `ey_driver` — Transverse Current Source for the Wave Equation

8.1 Physical Setup

The `ey_driver` injects a transverse EM wave by acting as an external current source in the wave equation. As established in Section 5.4, the source term `djy_array` enters the normalized wave equation (27) as $\tilde{S} = j_{\text{ext},y}/(n_0 e v_0)$.

The driver is parametrized by a vector potential amplitude $\tilde{a}_0$, wavenumber $\tilde{k}$, and frequency $\tilde{\omega}$, describing the desired EM wave. We need to determine what external current $\tilde{S}$ to apply.

8.2 Derivation: Required External Current

Suppose the driver acts in a localized region (controlled by the envelope function) and we want to sustain a wave of the form $\tilde{a}_{\text{drv}} = \tilde{a}_0 \sin\left(\tilde{k}\tilde{x} - \tilde{\omega}\tilde{t}\right)$ within that region. Substituting into the wave equation (27), this component satisfies:

$$\underbrace{-\tilde{\omega}^2 \tilde{a}_{\text{drv}}}_{\partial_{\tilde{t}}^2 \tilde{a}_{\text{drv}}} = \underbrace{-c_{\text{norm}}^2 \tilde{k}^2 \tilde{a}_{\text{drv}}}_{c^2 \partial_{\tilde{x}}^2 \tilde{a}_{\text{drv}}} - \frac{n_e}{n_0} \tilde{a}_{\text{drv}} + \tilde{S} \tag{46}$$

Solving for $\tilde{S}$:

$$\tilde{S} = \left(-\tilde{\omega}^2 + c_{\text{norm}}^2 \tilde{k}^2 + \frac{n_e}{n_0}\right) \tilde{a}_0 \sin\left(\tilde{k}\tilde{x} - \tilde{\omega}\tilde{t}\right) \tag{47}$$

If $(\tilde{\omega}, \tilde{k})$ satisfy the plasma dispersion relation $\tilde{\omega}^2 = c_{\text{norm}}^2 \tilde{k}^2 + n_e/n_0$, then $\tilde{S} = 0$: the wave propagates freely with no source needed.

8.3 Interpretation

Equation (47) has a clear physical meaning: $\tilde{S}$ is the external transverse current density (normalized to $n_0 e v_0$) required to sustain the desired wave against the plasma's dielectric response.

However, in practice the wave solver *already computes* the $c_{\text{norm}}^2 \partial_{\tilde{x}}^2 \tilde{a}$ and $-(n_e/n_0)\tilde{a}$ terms self-consistently from the total field $\tilde{a}$. If we include those terms in $\tilde{S}$ as well, the driven component would be double-counted: the solver would apply the spatial derivative and density coupling to the total $\tilde{a}$ (which includes the driven part), and we would also be supplying them via $\tilde{S}$.

Therefore, the correct source to supply is only the "inertial" part—the piece that the wave solver does *not* compute from the field:

$$\boxed{\tilde{S}_{\text{ey,driver}} = -\tilde{\omega}^2 \cdot \tilde{a}_0 \cdot \sin\left(\tilde{k}\tilde{x} - \tilde{\omega}\tilde{t}\right)} \tag{48}$$

This is equivalent to saying: the source injects $\partial_{\tilde{t}}^2 \tilde{a}_{\text{drv}}$ directly, and the wave solver handles the rest ($c^2 \partial_x^2$ and density coupling) self-consistently from whatever $\tilde{a}$ field results.

Physical reading: This source represents an antenna current that "pushes" the vector potential at the specified frequency. The resulting wave amplitude in the plasma depends on how $(\tilde{\omega}, \tilde{k})$ relates to the local dispersion relation:

- If $\tilde{\omega}^2 \approx c_{\text{norm}}^2 \tilde{k}^2 + n_e/n_0$ (on the dispersion surface), the self-consistent terms nearly cancel $\tilde{S}$ and the wave is strongly resonantly driven—small forcing produces large amplitude. In practice, the envelope localization and absorbing boundaries prevent true divergence.

- If $\tilde{\omega}^2$ is far from the dispersion surface, the forcing competes with a large restoring term and the effective amplitude is reduced.

In either case, $\tilde{a}_0$ as specified in the config is the *source strength*, not directly the resulting wave amplitude (the relationship between the two depends on the plasma conditions and envelope shape).

8.4 Comparison with Current Code

The current `Driver.get_this_pulse` returns $+|\tilde{k}| \cdot \tilde{a}_0 \cdot \sin(\ldots)$ for both driver types.

	Current Code	**Correct**
Prefactor	$\lvert\tilde{k}\rvert \cdot \tilde{a}_0$	$\tilde{\omega}^2 \cdot \tilde{a}_0$
Sign	$+$	$-$

Discrepancy: The `ey_driver` has wrong prefactor ($|\tilde{k}|$ vs $\tilde{\omega}^2$) and wrong sign.

9 Summary

9.1 Components Verified Correct

Component	Description	Status
Poisson solver	Spectral solve of $\partial^2\tilde{\phi}/\partial\tilde{x}^2 = -\tilde{\rho}$	Correct
Ampere solver	Euler step $\tilde{E}^{n+1} = \tilde{E}^n - \Delta\tilde{t}\,\tilde{j}$	Correct
Hampere solver	Exact Hamiltonian integration in Fourier space	Correct
Wave equation structure	$\partial_{\tilde{t}}^2\tilde{a} = c_{\text{norm}}^2\partial_{\tilde{x}}^2\tilde{a} - (n_e/n_0)\tilde{a} + \tilde{S}$	Correct
n_e term in wave eq.	Time-averaged $\frac{1}{2}(n_e^n + n_e^{n+1})$	Correct
`edfdv` push	Shift by $(\tilde{q}_s/\tilde{m}_s)\tilde{E}\Delta\tilde{t}$ per species	Correct

9.2 Bugs Found

Component	Current Code	Correct	Severity
Pond. force	$-\frac{1}{2}\partial_{\tilde{x}}(\tilde{a}^2)$ added as $\tilde{E}$; wrong-sign accel.	$-\frac{\tilde{q}_s}{2\tilde{m}_s}\partial_{\tilde{x}}(\tilde{a}^2)$ (species-dep. $\tilde{E}$)	Sign flip
`ex_driver`	$\lvert\tilde{k}\rvert \cdot \tilde{a}_0$	$\tilde{\omega} \cdot \tilde{a}_0$	$\times\, c_{\text{norm}}$
`ey_driver`	$+\lvert\tilde{k}\rvert \cdot \tilde{a}_0$	$-\tilde{\omega}^2 \cdot \tilde{a}_0$	wrong

9.3 Recommended Fixes

9.3.1 Ponderomotive force (multi-species correct)

Move the ponderomotive contribution inside the per-species loop in `edfdv`, or compute a per-species effective field:

```
# In ElectricFieldSolver or the calling code:
grad_a2 = jnp.gradient(a**2.0, self.dx)[1:-1]

# Per species: E_pond_s = -(tq_s / (2 * tm_s)) * grad(a^2)
# For electron-only shortcut: E_pond_e = +0.5 * grad(a^2)
```

9.3.2 Driver classes

Split into two classes with correct normalization:

```
class ExDriver:
    """Electric␣field␣driver␣for␣the␣Vlasov␣equation."""
    def get_this_pulse(self, this_pulse, current_time):
        kk = this_pulse.k0
        ww = this_pulse.w0 + this_pulse.dw0
        factor = this_pulse.envelope(self.xax, current_time)
        return factor * ww * this_pulse.a0 \
               * jnp.sin(kk * self.xax - ww * current_time)
```

```
class EyDriver:
    """Transverse␣current␣source␣for␣the␣wave␣equation."""
    def get_this_pulse(self, this_pulse, current_time):
        kk = this_pulse.k0
        ww = this_pulse.w0 + this_pulse.dw0
        factor = this_pulse.envelope(self.xax, current_time)
        return -factor * ww**2 * this_pulse.a0 \
               * jnp.sin(kk * self.xax - ww * current_time)
```